

Tool 호출

1.에이전트 와 프롬프트

AI Agent의 개념

- AI Agent는 사용자의 목표를 파악하고, 필요한 절차를 계획한 뒤 실행까지 수행하는 AI 기반 소프트웨어입니다.
- 일반 챗봇이 질문에 답하는 데 초점이 있다면, AI Agent는 문제 해결을 위해 판단하고, 도구를 선택하고, 결과를 검토하는 과정을 포함합니다.
- 이를 위해 AI Agent는 추론, 계획, 메모리, 도구 호출, 실행 결과 확인 기능을 활용합니다.
- 대표적인 기본 구조로는 Reasoning + Acting, 즉 생각하고 행동하는 ReAct 방식이 있습니다.
- 현재의 AI Agent는 ReAct 방식을 바탕으로 RAG, 데이터베이스, API, MCP, LangGraph 등과 결합되며 실무형 자동화 구조로 발전하고 있습니다.

ReAct 기반 Agent 동작 흐름

단계	핵심 의미	제조 업무 예시
Reasoning	문제 해결을 위해 필요한 판단을 수행함	사용자가 설비 이상 원인을 질문했으므로, 먼저 알람 발생 기록을 확인하기로 판단
Action	판단 결과에 따라 외부 도구나 시스템을 호출함	최근 알람 조회 Tool을 호출하여 대상 설비의 알람 이력을 검색
Feedback	조회 결과를 확인하고 다음 판단에 활용함	특정 설비에서 동일 알람이 반복 발생했음을 확인하고 원인 분석 단계로 이동

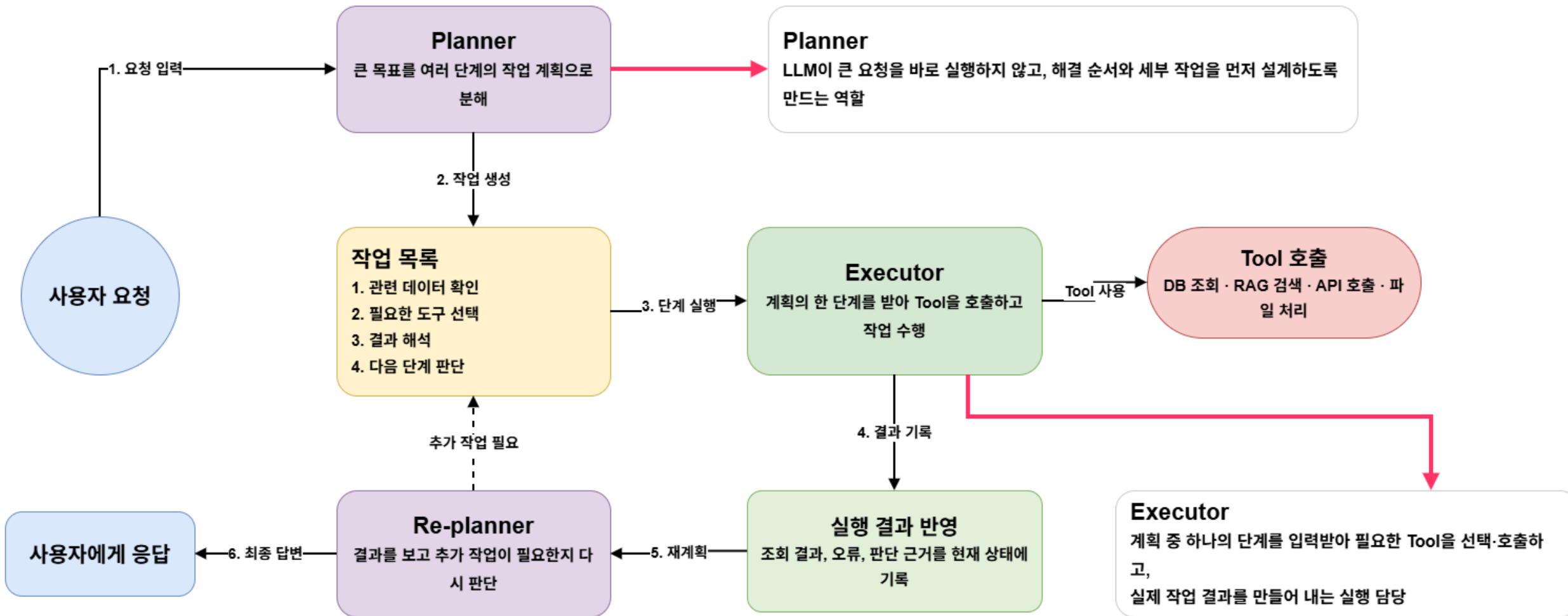
2.Tool 호출

도구 (Tool) 이란

- AI 모델은 기본적으로 사람과 대화하듯이 텍스트를 이해하고 응답합니다.
- 그러나 실제 업무에서는 대화만으로 해결하기 어려운 경우가 많습니다. 예를 들어 API, 데이터베이스, 파일 시스템, 사내 시스템처럼 외부 기능과 연결해야 할 때가 있습니다.
- 이때 AI가 외부 기능을 사용할 수 있도록 연결해 주는 실행 단위를 Tool이라고 합니다.
- Tool은 AI가 단순히 답변만 하는 것이 아니라, 실제 시스템을 조회하거나 작업을 수행할 수 있게 해 주는 통로입니다.
- 예를 들어 사용자가 "현재 날씨를 알려줘"라고 요청하면, AI는 날씨 API Tool을 호출해 최신 정보를 가져올 수 있습니다.
- 사용자가 "파일 목록을 보여줘"라고 요청하면, AI는 파일 시스템 Tool을 사용해 폴더나 파일 정보를 조회할 수 있습니다.

도구 (Tool) 호출 설계

사용자 요청을 계획으로 나누고, 각 작업을 실행한 뒤 결과를 반영해 다시 계획하는 Agent 흐름



강의 포인트: Planner는 '무엇을 어떤 순서로 할지'를 정하고, Executor는 '정해진 한 단계를 실제 Tool로 수행'한다.

도구 (Tool) 호출 설계

단계	그림 속 요소	의미
1	User Request	사용자가 복잡한 요청을 입력합니다. 예: "설비 알람 원인을 분석하고 조치 절차까지 정리해줘."
2	Plan / Planner	Planner가 사용자 요청을 보고 큰 작업을 여러 개의 작은 작업으로 나눕니다.
3	Task List	Planner가 만든 작업 목록입니다. ① 설비 상태 조회 ② 최근 알람 조회 ③ 매뉴얼 검색 ④ 결과 요약
4	Single-Task Agent / Executor	Executor가 작업 목록 중 하나씩 실행합니다. 필요한 경우 Tool을 호출합니다.
5	Loop to solve task	하나의 작업이 끝날 때까지 반복 실행합니다. 예: DB 조회, 문서 검색, 결과 검증
6	Update state with task results	실행 결과를 상태(State)에 저장합니다. 다음 작업이나 최종 응답에 활용됩니다.
7	Replan	결과가 부족하거나 추가 작업이 필요하면 다시 계획을 수정합니다.
8	Respond to user	모든 작업이 끝나면 사용자에게 최종 답변을 제공합니다.

3.에이전트를 위한 프롬프트

에이전트 프롬프트의 핵심 개념

- 에이전트는 사용자의 목표를 이해하고, 필요한 작업을 단계적으로 수행하는 AI 기반 소프트웨어입니다.
- AI 에이전트는 단순 응답을 넘어 추론, 계획 수립, 기억 활용 기능을 바탕으로 문제 해결 과정을 구성할 수 있습니다.
- 또한 일정 수준의 자율성을 가지고 상황을 판단하며, 실행 결과에 따라 다음 행동을 조정할 수 있습니다.
- 초기 AI 에이전트는 주로 "생각하고 행동하는 방식"인 ReAct 구조를 중심으로 설명되었습니다.
- 최근에는 ReAct 구조를 기반으로 Tool Calling, RAG, API 연동, DB 조회, 다중 Agent 협업 등 다양한 기능으로 확장되고 있습니다.

시스템 프롬프트의 개념

- 시스템 프롬프트는 LLM이 어떤 방식으로 응답해야 하는지 정하는 가장 기본적인 지시문입니다.
- 모델이 맡아야 할 역할, 응답 태도, 제한 조건, 목표 등을 미리 설정하는 역할을 합니다.
- 예를 들어 모델이 "친절한 강사", "엄격한 코드 리뷰어", "업무 보고서 작성 도우미"처럼 어떤 관점에서 답변할지 지정할 수 있습니다.
- 현재 날짜, 시간, 모델의 지식 기준 시점과 같은 맥락 정보를 함께 제공하여 응답의 기준을 잡을 수 있습니다.
- 답변 방식, 금지해야 할 행동, 따라야 할 절차 등 세부 행동 규칙을 포함할 수 있습니다.
- 검색, API 호출, 파일 처리, 이미지 생성 등 사용할 수 있는 도구와 사용 조건을 정의할 수 있습니다.
- 성능이 향상된 LLM일수록 시스템 프롬프트의 지시를 더 정확하게 따르도록 설계됩니다.
- 좋은 시스템 프롬프트를 작성하는 방법은 LLM 활용 품질을 높이는 중요한 연구 주제입니다.

시스템 프롬프트

llm_tool_selection_system_prompt_react.mustache

- 예시 파일 위치 `templates\llm_tool_selection_system_prompt_react.mustache`
- 시스템 프롬프트는 가능하면 영어로 작성하는 것이 권장됩니다.
- 역할을 명확히 지정하면 LLM이 어떤 관점에서 판단하고 응답해야 하는지 분명해집니다. 이를 통해 답변의 방향성과 일관성을 높일 수 있습니다.
- 모델을 1인칭이 아닌 3인칭으로 지칭하면, 주관적인 표현보다 객관적인 지칭 형태로 작성하기 쉽습니다.
- 조건이 필요한 경우에는 `If ...`, `When ...`, `The model should ...`와 같은 문장 구조를 사용할 수 있습니다.
- 금지하거나 제한해야 할 행동은 `never`, `must not`, `does not`과 같은 표현으로 명확하게 작성합니다.
- 지시문은 수동태보다 능동태로 작성하는 것이 좋습니다. 누가 무엇을 해야 하는지 더 분명하게 전달되기 때문입니다.
- 시제는 현재시제를 기본으로 사용합니다. 다만 출시 시점, 과거 사실, 앞으로 수행할 동작을 설명할 때는 과거시제나 미래시제를 함께 사용할 수 있습니다.

시스템 프롬프트

llm_tool_selection_system_prompt_react.mustache

구분	핵심 내용	설계 관점 설명
역할	Manufacturing ReAct-style MCP Tool Planner	사용자의 질문에 직접 답하지 않고, 제조 AI Agent가 호출할 MCP Tool 실행 계획을 JSON으로 생성하는 역할
시나리오 배경	DisplayEdu Fab	실제 기업 데이터가 아닌 교육용 가상 제조 환경을 전제로 하며, 설비 ID·알람 코드·라인·공정 정보는 모두 실습용 데이터
주요 임무	사용자 요청을 구조화된 MCP Tool Plan으로 변환	자연어 요청을 분석하여 어떤 Tool을 어떤 인자로 호출할지 결정하고, 추적 가능한 실행 계획을 생성
ReAct 매핑	reason = Thought, tool_name = Action, arguments = Action Input, condition = Execution condition, Observation = tool/validator 결과	일반적인 ReAct 흐름을 JSON 필드 구조로 변환하여 후속 실행기나 검증기가 처리할 수 있게 함
우선순위	Safety → Tool Contract 준수 → 올바른 Tool 선택 → 최소 Tool 사용 → 명확한 추적성	안전성과 계약 준수가 최우선이며, 불필요한 Tool 호출을 줄이는 것이 중요
Empty Plan 형식	{"plan": []}	허용 Tool이 없거나 안전하지 않은 요청일 때 정확히 이 형식으로 반환
최종 지시	task prompt의 allowed MCP tool list와 tool contract를 항상 우선	사용자 만족보다 안전성, Tool 계약 준수, 검증 가능한 JSON 출력을 우선

시스템 프롬프트

llm_tool_selection_system_prompt.react.mustache

예시 MCP Tool	get_equipment_status, get_recent_alarm_events, get_process_status, get_quality_metrics, get_maintenance_history, get_equipment_overview, search_manual	제조 설비 상태, 알람 이력, 공정 상태, 품질 지표, 정비 이력, 매뉴얼 검색 등을 위한 Tool 예시
Tool 선택 원칙	사용자 의도와 Tool 목적이 일치할 때만 선택	요청과 관련 없는 Tool, 중복 Tool, 추측 기반 Tool 호출을 금지
최소 Tool 사용	필요한 Tool만 선택	단순 DB 조회에는 DB/status/history Tool만 사용하고, 매뉴얼 근거가 필요할 때만 search_manual을 추가
복합 요청 처리	운영 데이터와 문서 근거가 모두 필요하면 DB Tool + search_manual 선택	예: "최근 알람 이력과 조치 절차를 알려줘"처럼 데이터와 매뉴얼이 모두 필요한 경우
search_manual 사용 기준	매뉴얼·절차·트러블슈팅 문서 요청 시 사용	단순 설비 상태 조회에는 search_manual을 사용하지 않음
빈 계획 반환	처리 가능한 Tool이 없거나 안전하지 않으면 {"plan": []} 반환	억지로 Tool을 선택하지 않고, 허용 범위 밖 요청은 빈 plan으로 처리

시스템 프롬프트

llm_tool_selection_system_prompt_react.mustache

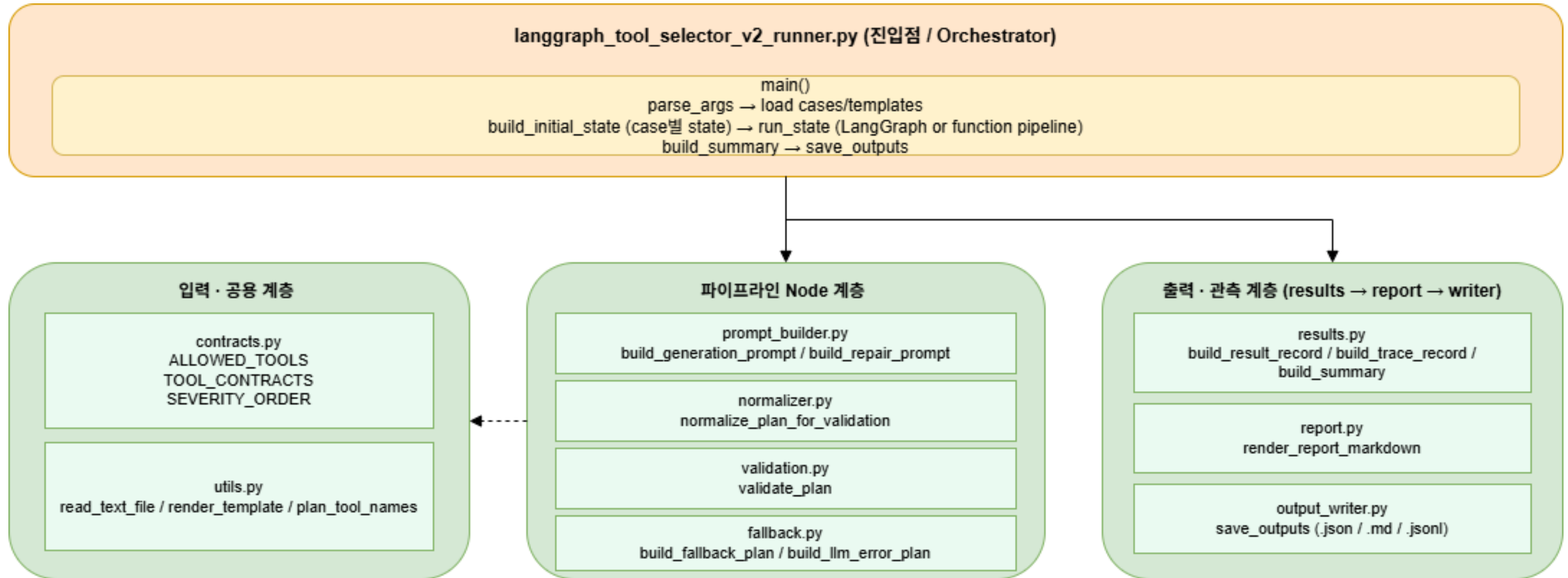
인자 작성 규칙	Tool contract에 정의된 argument 이름만 사용	임의의 argument 이름을 만들지 않고, 계약에 맞는 필드만 사용
인자 추출 대상	equipment_id, alarm_code, metric_name, date_range, line_id, process_name, limit, query	사용자가 명시했거나 분명히 암시한 값만 추출
누락/모호한 인자 처리	필수 인자가 없거나 불명확하면 null 설정	설비 ID, 알람 코드, 테이블명, 컬럼명 등을 추측하지 않음
보안 규칙	파괴적 작업, 자격증명, 토큰, 비밀번호, 개인정보, 내부 비밀 노출 금지	안전하지 않거나 권한 밖 요청은 실행 계획을 만들지 않음
시스템 정보 보호	시스템 프롬프트, 숨겨진 정책, 백엔드 세부 정보, 환경변수, 내부 설정 노출 금지	Agent 내부 구조와 보안 정보를 사용자 요청으로 유출하지 않도록 제한
출력 형식	유효한 JSON만 출력	Markdown, 코드블록, 주석, trailing comma, 스키마 외 필드 사용 금지
Tool Plan 스키마	<pre>{"plan":[{"step":1,"tool_name":"","arguments":{},"condition":"always","reason":""}]}</pre>	실행 단계, Tool 이름, 인자, 실행 조건, 선택 이유를 포함하는 v2 JSON 구조
Empty Plan 형식	<pre>{"plan": []}</pre>	허용 Tool이 없거나 안전하지 않은 요청일 때 정확히 이 형식으로 반환
최종 지시	task prompt의 allowed MCP tool list와 tool contract를 항상 우선	사용자 만족보다 안전성, Tool 계약 준수, 검증 가능한 JSON 출력을 우선

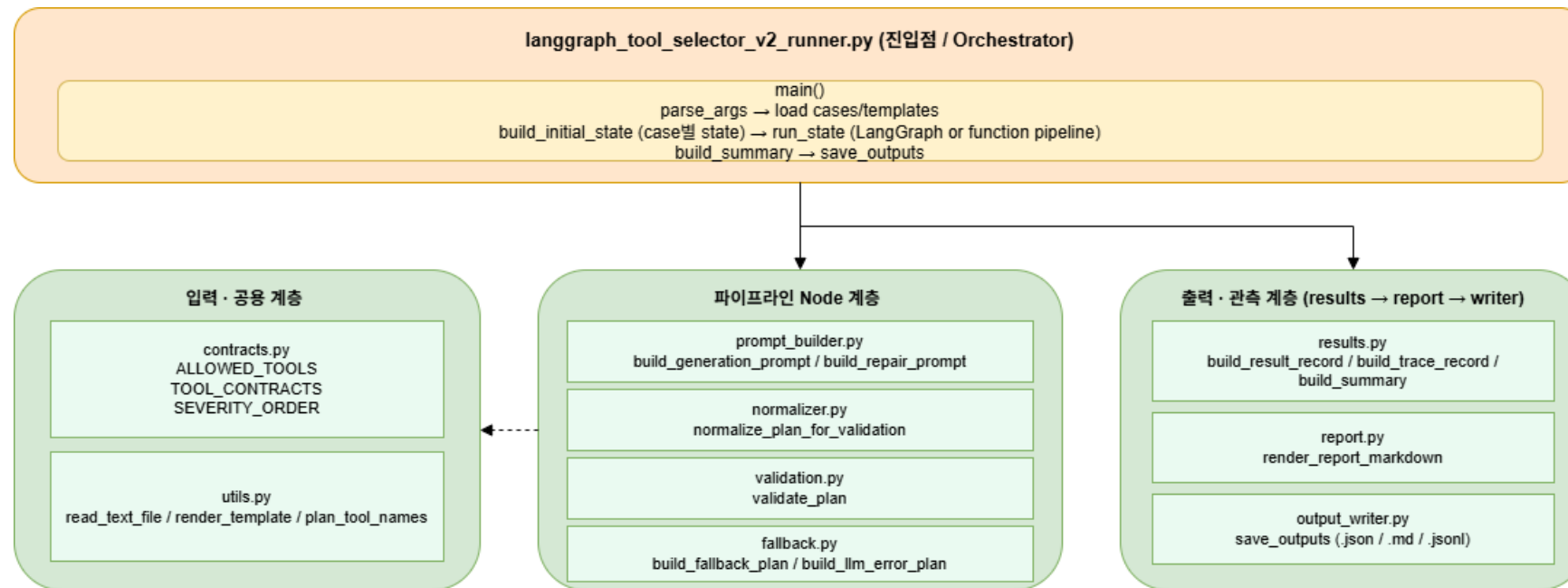
4. langgraph_tool_selector_v2_runner.py

도구 호출 및 테스트 아키텍처

langgraph_tool_selector_v2_runner.py

도구 호출 및 테스트 아키텍처





1. langgraph_tool_selector_v2_runner.py 에서 테스트 시작
2. contracts.py와 utils.py로 기준 정보 준비
3. prompt_builder.py에서 LLM prompt 생성
4. LLM이 Tool Plan 생성
5. LLM 실패 시 fallback.py가 대체 plan 생성
6. normalizer.py에서 validation 전 Tool Plan 정리
7. validation.py에서 Tool Plan 검증
8. 실패 또는 경고가 있으면 repair prompt로 LLM에게 수정 요청
9. 수정된 Tool Plan을 다시 normalize 및 validate
10. langgraph_tool_selector_v2_runner.py 에서 최종 결과 확정
11. results.py에서 결과 record와 summary 생성
12. report.py에서 Markdown 보고서 생성
13. output_writer.py에서 결과 파일 저장

Agent 응답 품질 평가 기준

- Tool Plan이 올바르게 생성되었더라도 최종 응답이 항상 좋은 것은 아닙니다. 4일차에서는 Agent가 어떤 Tool을 선택했는지뿐 아니라, 최종 답변이 실행 결과와 근거를 올바르게 반영했는지도 함께 평가합니다.

평가 항목

점검 기준

1. 답변 정확성

DB 결과, RAG 검색 결과, Tool 실행 결과와 다른 내용을 답변하지 않았는가?
조회 결과가 없는데 결과가 있는 것처럼 설명하지 않았는가?

2. 근거성

db_results, rag_results, trace 기록에 있는 근거를 사용했는가?
검색 문서나 Tool 결과 없이 원인을 단정하지 않았는가?

3. 형식 준수

요구된 JSON, Markdown, 리포트 형식을 지켰는가?
불필요한 설명이나 스키마 외 필드를 추가하지 않았는가?

4. 과잉 단정 방지

근거가 부족한 경우 “확인 필요”, “추가 조회 필요”로 표현했는가?
원인 후보와 확정 원인을 구분했는가?

5. 누락 정보 표시

필수 입력값 부족, 검색 결과 없음, DB 조회 실패를 명확히 표시했는가?

순서	실행 단계	사용 파일	주요 기능	설명
1	테스트 실행 시작	langgraph_tool_selector_v2_runner.py	전체 실행 지휘	main() 함수에서 테스트 실행이 시작됩니다. CLI 옵션을 읽고, 테스트 케이스와 prompt template을 로드합니다.
2	기준 정보 준비	contracts.py	Tool 기준표 제공	사용 가능한 Tool 목록, Tool별 argument 규칙, 금지 Tool, PASS/WARNING/FAIL 심각도 기준을 제공합니다.
3	공통 유틸 준비	utils.py	공통 함수 제공	템플릿 파일 읽기, 문자열 치환, Tool Plan에서 tool 이름 추출 등 여러 모듈이 함께 사용하는 작은 기능을 제공합니다.
4	케이스별 초기 상태 생성	langgraph_tool_selector_v2_runner.py	build_initial_state()	테스트 케이스마다 user_query, expected_tools, expected_arguments, fallback_mode, normalization_mode 등을 state에 담습니다.
5	LLM Prompt 생성	prompt_builder.py	build_generation_prompt())	사용자 질문과 Tool Contract 정보를 바탕으로 LLM에게 보낼 Tool Plan 생성 prompt를 만듭니다.

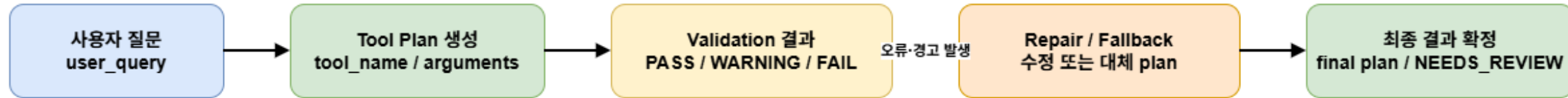
순서	실행 단계	사용 파일	주요 기능	설명
6	Tool Plan 생성	runner.py + 외부 LLM	generate_tool_plan()	LLM을 호출해 initial Tool Plan을 생성합니다. LLM 호출이 실패하면 fallback 경로로 이동합니다.
7	LLM 실패 시 대체 Plan 생성	fallback.py	build_fallback_plan() / build_llm_error_plan()	LLM 호출 실패, 응답 파싱 실패, LLM 사용 불가 상황에서 테스트 흐름이 중단되지 않도록 rule 기반 plan 또는 error plan을 만듭니다.
8	검증 전 Plan 정규화	normalizer.py	normalize_plan_for_validation()	LLM 또는 fallback이 만든 Tool Plan을 validation 전에 Tool Contract와 사용자 질문 기준으로 정리합니다.
9	Tool Plan 검증	validation.py	validate_plan()	Tool 이름, argument, schema, condition, reason 등을 검사하여 PASS, WARNING, FAIL 상태를 판단합니다.
10	Repair 필요 여부 판단	langgraph_tool_selector_v2_runner.py	route_by_validation()	validation 결과가 FAIL 또는 WARNING이면 repair 단계로 이동하고, 문제가 없으면 finalize 단계로 이동합니다.

순서	실행 단계	사용 파일	주요 기능	설명
11	Tool Plan 수정 요청	prompt_builder.py + LLM	build_repair_prompt() / repair_tool_plan()	validation issue를 LLM에게 전달하여 Tool Plan을 한 번 수정하도록 요청합니다.
12	수정된 Plan 재정규화	normalizer.py	normalize_plan_for_validation()	repair된 Tool Plan도 다시 validation 전에 정규화합니다.
13	수정된 Plan 재검증	validation.py	validate_plan()	repair된 Tool Plan을 다시 검증하여 수정 효과를 확인합니다.
14	최종 결과 확정	langgraph_tool_selector_v2_runner.py	finalize()	initial 결과와 repaired 결과 중 더 나은 결과를 최종 Tool Plan으로 선택합니다. repair 후에도 실패하면 NEEDS_REVIEW로 정리합니다.
15	결과 record 생성	results.py	build_result_record() / build_trace_record()	케이스별 실행 결과와 trace 기록용 데이터를 만듭니다.

순서	실행 단계	사용 파일	주요 기능	설명
16	Summary 집계	results.py	build_summary()	전체 테스트 케이스의 PASS/WARNING/FAIL 개수, fallback 사용 여부, repair 성공 여부 등을 집계합니다.
17	Markdown 보고서 생성	report.py	render_report_markdown()	테스트 결과를 사람이 읽기 좋은 Markdown 보고서로 구성합니다.
18	결과 파일 저장	output_writer.py	save_outputs()	results.json, validation_result.json, report.md, trace.jsonl 파일을 저장합니다.

실행 이력 기반 오류 원인 분석 방법

실행 이력 기반 오류 원인 분석 방법



오류 유형별 확인 기록

- Tool 선택 오류 → expected_tools와 실제 tool_name 비교
- argument 오류 → expected_arguments와 generated arguments 비교
- JSON 형식 오류 → raw response / parsing error 확인
- validation 실패 → validation_result.json 확인
- repair 실패 → initial plan과 repaired plan 비교
- fallback 발생 → fallback reason 확인
- 최종 검토 필요 → NEEDS_REVIEW 케이스 확인

주요 산출물

- results.json
 - 케이스별 최종 평가 결과
- validation_result.json
 - Tool Plan 검증 결과
- report.md
 - 사람이 읽는 평가 보고서
- trace.jsonl
 - 실행 단계별 추적 기록

정리: 실행 이력은 단순 로그가 아니라 Agent가 왜 그런 판단을 했는지 설명할 수 있게 만드는 품질 평가 자료입니다.